CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati
saradhi@iitg.ac.in

Tutorial #02 C++ Structures, Functions, File I/O

Overview

- Structures
- Functions
- File I/O

Structures

Structures - 01

- A structure is a collection of variables referenced under one name
- A structure declaration forms a template
- The variables in the structure are called members.
- Structures are user-defined data types

```
#include<iostream>

struct student
{
  int roll_number;
  char name[50];
  char email[50];
};
```

Structures - 02

- Now that a user-defined data type is created, we can have variables with this user-defined data type
- This program to the right shows the method of declaring a struct variable and initializing it.

```cpp
#include<iostream>
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
  char login[50];
};
int main(int argc, char *argv[])
{
  struct student s1;
  s1.roll_number = 220101999;
  strcpy(s1.name, "student-999");
  strcpy(s1.login, "s999");
  cout << "roll-number: " << s1.roll_number << endl;
  cout << "name       : " << s1.name << endl;
  cout << "login      : " << s1.login << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-01.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
roll-number: 220101999
name       : student-999
login      : s999
```

Structures - 03

- Now that a user-defined data type is created, we can have variables with this user-defined data type
- Direct initialization

```
#include<iostream>
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
  char login[50];
};
int main(int argc, char *argv[])
{
  struct student s1 = {220101999, "student-999", "s999"};
  cout << "roll-number: " << s1.roll_number << endl;
  cout << "name       : " << s1.name << endl;
  cout << "login      : " << s1.login << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-02.cc
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
roll-number: 220101999
name       : student-999
login      : s999
```

Structures - 04

- Pointers inside a structure
- Variable declaration: structure student s1;
- Initialization: s1.roll_number=220101999;
- Pointer: s1.ptr = &s1.roll_number;

```
#include<iostream>

struct student
{
  int roll_number;
  int *ptr;
};
```

Structures - 05

- Array of structures
- s1 is array of structures containing three "struct student" elements
- Data is initialized at the time of declaration

```
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
  char login[50];
};
int main(int argc, char *argv[])
{
  struct student s1[3] = {
    {220101999, "student-999", "s999"},
    {220101888, "student-888", "s888"},
    {220101777, "student-777", "s777"}
  };
  cout << "roll-number: " << s1[2].roll_number << endl;
  cout << "name       : " << s1[2].name << endl;
  cout << "login      : " << s1[2].login << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-03.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
roll-number: 220101777
name       : student-777
login      : s777
```

Structures - 06

- Array of structures
- s1 is array of structures containing three "struct student" elements
- Data is read from file "data.txt" using indirection operation

```cpp
#include<iostream>
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
  char login[50];
};
int main(int argc, char *argv[])
{
  struct student s1[3];
  cin >> s1[0].roll_number >> s1[0].name >> s1[0].login;
  cin >> s1[1].roll_number >> s1[1].name >> s1[1].login;
  cin >> s1[2].roll_number >> s1[2].name >> s1[2].login;
  cout << "roll-number: " << s1[2].roll_number << endl;
  cout << "name       : " << s1[2].name << endl;
  cout << "login      : " << s1[2].login << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-04.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
^C
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out < data.txt
roll-number: 220101777
name       : student-777
login      : s777
```

```
220101999 student-999 s999
220101888 student-888 s888
220101777 student-777 s777
```

Structures - 07

- Structure pointer
- s1 is a structure
- Variable ptr is a struct student pointer
- Method of accessing and assignment

```cpp
#include <iostream>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
};
int main(int argc, char *argv[])
{
    struct student s1 = {220101999, "student-999"};
    struct student* ptr = &s1;

    cout << "Name: " << ptr->name << ", roll_number: " << ptr->roll_number << endl;

    ptr->roll_number = 220101333;
    cout << "Updated roll_number: " << ptr->roll_number << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Name: student-999, roll_number: 220101999
Updated roll_number: 220101333
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Structures - 08

- Dynamic memory allocation of structures
- Using malloc

```cpp
#include<iostream>
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
};
int main(int argc, char *argv[])
{
    struct student* s1;
    s1 = (struct student *) malloc(10 * sizeof(struct student));
    s1[0].roll_number = 220101999;
    strcpy(s1[0].name, "student-999");
    cout << s1[0].roll_number << " " << s1[0].name << endl;
    free(s1);
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
220101999 student-999
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Structures - 09

- Dynamic memory allocation of structures

- Using new

```
#include<iostream>
#include<string.h>
using namespace std;
struct student
{
  int roll_number;
  char name[50];
};
int main(int argc, char *argv[])
{
    struct student* s1 = new struct student[10];
    s1[0].roll_number = 220101999;
    strcpy(s1[0].name, "student-999");
    cout << s1[0].roll_number << " " << s1[0].name << endl;
    delete[] s1;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-07.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
220101999 student-999
```

Structures - 10

- Structure within structure

```cpp
#include <iostream>
using namespace std;
struct hostel {
    char name[50];
    int room_number;
};

struct student {
    int roll_number;
    struct hostel h1;
};
int main(int argc, char *argv[])
{
    struct student s1 = {220101999, {"Dihing", 233}};
    cout << "roll_number: " << s1.roll_number << endl;
    cout << "Hostel name: " << s1.h1.name << endl;
    cout << "Hostel room number: " << s1.h1.room_number << endl;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-08.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
roll_number: 220101999
Hostel name: Dihing
Hostel room number: 233
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Structures - 11

- Array of pointers to structures

```
#include <iostream>
using namespace std;
struct student {
    int roll_number;
    char name[50];
};
int main(int argc, char *argv[])
{
    int i = 0;
    struct student s1 = {220101999, "student-999"};
    struct student s2 = {220101888, "student-888"};
    struct student s3 = {220101777, "student-777"};
    struct student *str_arr[3]  = {&s1, &s2, &s3};
    for( i = 0; i < 3; i++ )
    {
      cout << "roll number: " << str_arr[i]->roll_number << endl;
      cout << "name :" << str_arr[i]->name << endl;
    }
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-09.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
roll number: 220101999
name :student-999
roll number: 220101888
name :student-888
roll number: 220101777
name :student-777
```

Structures - 12

- Structures with member functions

```cpp
#include <iostream>
using namespace std;
struct student
{
  int roll_number;
  char name[50];

  void print()
  {
    cout << "Roll Number: " << roll_number << endl;
    cout << "Name       : " << name << endl;
  }
};

int main(int argc, char *argv[])
{
    struct student s1 = {220101999, "student-999"};
    s1.print();
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-struct-10.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Roll Number: 220101999
Name       : student-999
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Functions

Functions - 01

- Functions can be declared and called in various ways, depending on their purpose and usage.
- Simple Function with no arguments and no return value - Declaration and Call

```cpp
#include <iostream>
using namespace std;

void print(void)
{
  cout << "Hello world" << endl;
}

int main(int argc, char *argv[])
{
  print();
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-01.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Hello world
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Functions - 02

- Functions can be declared and called in various ways, depending on their purpose and usage.
- Simple Function with return value. Declaration and Call

```cpp
#include <iostream>
using namespace std;

int print(void)
{
  cout << "Hello world" << endl;
  return 100;
}

int main(int argc, char *argv[])
{
  cout << print() << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-02.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Hello world
100
```

Functions - 03

- Functions can be declared and called in various ways, depending on their purpose and usage.
- Simple Function with no return value and one argument. Declaration and Call
- Call by value

```cpp
#include <iostream>
using namespace std;

int square(int x)
{
  x = x * x;
  cout << "x: " << x << endl;
  return x;
}

int main(int argc, char *argv[])
{
  int y = 5;
  cout << "before calling square(y): " << y << endl;
  cout << "sqare: " << square(y) << endl;
  cout << "after calling square(y): " << y << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-03.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
before calling square(y): 5
sqare: x: 25
25
after calling square(y): 5
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Functions - 04

- Functions can be declared and called in various ways, depending on their purpose and usage.
- Simple Function with no return value and one argument. Declaration and Call
- Call by reference
- Create a call by reference by passing a pointer to an argument, instead of the argument itself

```cpp
#include <iostream>
using namespace std;

int square(int *x)
{
  *x = *x * *x;
  cout << "x: " << *x << endl;
  return *x;
}

int main(int argc, char *argv[])
{
  int y = 5;
  cout << "before calling square(y): " << y << endl;
  cout << "sqare: " << square(&y) << endl;
  cout << "after calling square(y): " << y << endl;
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-04.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
before calling square(y): 5
sqare: x: 25
25
after calling square(y): 25
```

Functions - 05

- Function with two arguments

```cpp
#include <iostream>
using namespace std;

void print(int a, int b)
{
  cout << "a: " << a << " b: " << b << endl;
}

int main(int argc, char *argv[])
{
  int x = 10;
  int y = 20;
  print(x, y);
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
a: 10 b: 20
```

Functions - 06

- Function overloading
- Create identical function name with varying signatures
- Same print function with varying signatures

```cpp
#include <iostream>
using namespace std;
void print(int a)
{
  cout << "a: " << a << endl;
}
void print(float a)
{
  cout << "a: " << a << endl;
}
void print(int a, int b)
{
  cout << "a: " << a << " b " << b << endl;
}
int main(int argc, char *argv[])
{
  int x = 10; int y = 20; float z = 5.2;
  print(x);
  print(z);
  print(x, y);
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
a: 10
a: 5.2
a: 10 b 20
```

Functions - 07

- Inline functions
- They do not involve function call overhead.

```cpp
#include <iostream>
using namespace std;
inline void print(int a)
{
  cout << "a: " << a << endl;
}
int main(int argc, char *argv[])
{
  int x = 10;
  print(x);
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-07.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
a: 10
```

Functions - 08

- Default value argument
- When the corresponding argument is not supplied, then function is invoked with default value

```cpp
#include <iostream>
using namespace std;
void print(int a = 10)
{
  cout << "a: " << a << endl;
}
int main(int argc, char *argv[])
{
  int x = 20;
  print(x);

  print();
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-08.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
a: 20
a: 10
```

Functions - 09

- Function pointers
- Declaring & calling function pointers

```cpp
#include <iostream>
using namespace std;
void greet()
{
    cout << "Hello, World!" << endl;
}
int add(int a, int b)
{
    return a + b;
}
int main(int argc, char *argv[])
{
    void (*greetPtr)() = greet;
    greetPtr();
    int (*addPtr)(int, int) = add;
    int result = addPtr(3, 4);
    cout << "Sum: " << result << endl;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-09.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Hello, World!
Sum: 7
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Functions - 10

- Array of function pointers
- Declaring & calling function pointers

```cpp
#include <iostream>
using namespace std;
int add(int a, int b) {
    return a + b;
}
int subtract(int a, int b) {
    return a - b;
}
int multiply(int a, int b) {
    return a * b;
}
int main(int argc, char *argv[])
{
    int (*operation[3])(int, int) = { add, subtract, multiply };
    int x = 10, y = 5;
    for (int i = 0; i < 3; i++)
    {
        cout << "Result: " << operation[i](x, y) << endl;
    }
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-10.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Result: 15
Result: 5
Result: 50
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Uses of Function Pointers

Uses of Function Pointers - 01

- **Implementing callbacks:** a function is passed as an argument to another function
- Dynamic dispatch of functions
- Creating tables of functions
- Encapsulating functions in data structures

```cpp
#include <iostream>
using namespace std;
void onEvent()
{
    cout << "Event triggered!" << endl;
}

void registerCallback(void (*callback)())
{
    cout << "Registering callback..." << endl;
    callback();
}

int main(int argc, char *argv[])
{
    registerCallback(onEvent);
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-10-a.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Registering callback...
Event triggered!
```

Uses of Function Pointers - 02

- Implementing callbacks:
- **Dynamic dispatch of functions**: dynamically choose which function to execute at runtime
- Creating tables of functions
- Encapsulating functions in data structures

```cpp
#include <iostream>
using namespace std;
void add(int a, int b)
{
    cout << "Sum: " << (a + b) << endl;
}
void subtract(int a, int b)
{
    cout << "Difference: " << (a - b) << endl;
}
int main(int argc, char *argv[])
{
    void (*operation)(int, int);
    char op;
    cout << "Enter operation (+ or -): ";
    cin >> op;
    if (op == '+')
    {
        operation = add;
    }
    else if (op == '-')
    {
        operation = subtract;
    }
    else
    {
        cout << "Invalid operation!" << endl;
        return 1;
    }
    operation(10, 5);
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-10-b.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Enter operation (+ or -): +
Sum: 15
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Enter operation (+ or -): -
Difference: 5
```

Uses of Function Pointers - 03

- Implementing callbacks:
- Dynamic dispatch of functions
- **Creating tables of functions**: allowing for efficient function lookups and execution
- Encapsulating functions in data structures

```cpp
#include <iostream>
using namespace std;
void add(int a, int b)
{
    cout << "Sum: " << (a + b) << endl;
}
void subtract(int a, int b)
{
    cout << "Difference: " << (a - b) << endl;
}
void multiply(int a, int b)
{
    cout << "Product: " << (a * b) << endl;
}
int main(int argc, char *argv[])
{
    void (*operations[3])(int, int) = { add, subtract, multiply };
    int choice;
    cout << "Enter operation (0: add, 1: subtract, 2: multiply): ";
    cin >> choice;
    if (choice < 0 || choice > 2)
    {
        cout << "Invalid choice!" << endl;
        return 1;
    }
    operations[choice](10, 5);
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-10-c.cc
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Enter operation (0: add, 1: subtract, 2: multiply): 0
Sum: 15
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Enter operation (0: add, 1: subtract, 2: multiply): 1
Difference: 5
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Enter operation (0: add, 1: subtract, 2: multiply): 2
Product: 50
```

Uses of Function Pointers - 04

- Implementing callbacks:
- Dynamic dispatch of functions
- Creating tables of functions
- **Encapsulating functions in data structures:** Function pointers can be encapsulated within data structures

```cpp
#include <iostream>
using namespace std;
struct Operation
{
    void (*func)(int, int);
};
void add(int a, int b)
{
    cout << "Sum: " << (a + b) << endl;
}
void subtract(int a, int b)
{
    cout << "Difference: " << (a - b) << endl;
}
int main(int argc, char *argv[])
{
    Operation op;
    op.func = add;
    op.func(10, 5);

    op.func = subtract;
    op.func(10, 5);
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-10-d.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Sum: 15
Difference: 5
```

Recursive functions

Recursive Functions

- A function calling itself with appropriate terminating condition

```cpp
#include <iostream>
using namespace std;

int factorial(int n)
{
    if (n <= 1)
      return 1;
    else
      return n * factorial(n - 1);
}

int main(int argc, char *argv[])
{
    cout << "Factorial of 5: " << factorial(5) << endl;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-11.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Factorial of 5: 120
```

Lambda functions

Lambda Functions - 01

- An anonymous function that can be defined within the scope of another function
- No need to define a full-fledged function
- lambda functions provide a concise way to create simple function objects without the need to define a full-fledged function

```
[ capture-list ] ( parameters ) -> return-type
{
    body
}
```

- **Capture-list**: Specifies which variables from the surrounding scope are accessible within the lambda function.
- **Parameters**: The parameters passed to the lambda function.
- **Return-type**: The return type of the lambda function. It can often be inferred by the compiler, so it's optional.
- **Body**: The code to be executed when the lambda function is called.

Lambda Functions - 02

- An anonymous function that can be defined within the scope of another function
- No need to define a full-fledged function
- lambda functions provide a concise way to create simple function objects without the need to define a full-fledged function

```cpp
#include <iostream>
using namespace std;
int main()
{
    auto greet = []()
    {
        cout << "Hello, World!" << endl;
    };

    greet();

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-14.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Hello, World!
```

Lambda Functions - 03

- Lambda function accepting two parameters

```cpp
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    auto add = [](int a, int b) -> int
    {
        return a + b;
    };

    int result = add(3, 4);
    cout << "result: " << result << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-15.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
result: 7
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

Lambda Functions - 04

- Lambda function accepting two capturing variables

```cpp
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int x = 10;
    int y = 20;

    auto add = [x, y]() -> int
    {
        return x + y;
    };

    cout << "result: " << add() << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-16.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
result: 30
```

Lambda Functions - 05

- Lambda function accepting two capturing variables passed as reference

```cpp
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    int x = 10;
    int y = 20;

    auto add = [&x, &y]() -> int
    {
        return x + y;
    };

    x = 15;
    cout << "result: " << add() << endl;

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-function-17.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
result: 35
```

File I/O

File I/O - 01

- Include the header <fstream> in the program to perform file I/O.
- It defines ifstream, ofstream, and fstream classes.
- Reading input from a file "data.txt" example

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(int argc, char *argv[])
{
    ifstream inFile("data.txt");
    int roll_number; char name[50], login[50];
    if( inFile.is_open() )
    {
        while( true )
        {
            inFile >> roll_number >> name >> login ;
            if( inFile.fail() )
            {
              break;
            }
            cout << "Roll Number: " << roll_number << endl;
            cout << "name       : " << name << endl;
            cout << "login      : " << login << endl;
        }
        inFile.close();
    }
    else
    {
        cerr << "Unable to open file for reading." << endl;
    }
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-02.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Roll Number: 220101999
name       : student-999
login      : s999
Roll Number: 220101888
name       : student-888
login      : s888
Roll Number: 220101777
name       : student-777
login      : s777
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

File I/O - 02

- Include the header <fstream> in the program to perform file I/O.
- It defines ifstream, ofstream, and fstream classes.
- Reading input from a file "data.txt" example

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-01.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Roll Number: 220101999
name       : student-999
login      : s999
Roll Number: 220101888
name       : student-888
login      : s888
Roll Number: 220101777
name       : student-777
login      : s777
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(int argc, char *argv[])
{
    ifstream inFile("data.txt");
    int roll_number; char name[50], login[50];
    if( inFile.is_open() )
    {
        while( inFile >> roll_number >> name >> login )
        {
            cout << "Roll Number: " << roll_number << endl;
            cout << "name       : " << name << endl;
            cout << "login      : " << login << endl;
        }
        inFile.close();
    }
    else
    {
        cerr << "Unable to open file for reading." << endl;
    }
    return 0;
}
```

File I/O - 03

- File writing to a file
- Default
- std::ios::out: Open for output operation
- std::ios::trunc: If the file already exists, its content is truncated (deleted) before opening.

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char *argv[])
{
    ofstream outFile("example.txt");
    if( outFile.is_open())
    {
        outFile << "Hello, World!" << endl;
        outFile << "This is a test file." << endl;
        outFile.close();
    }
    else
    {
        cerr << "Unable to open file for writing." << endl;
    }

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-03.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ more example.txt
Hello, World!
This is a test file.
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

File I/O - 04

- Opening file in append mode
- Every output line will be appended to the existing file contents.
- If a file already exists, the output will append.

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char *argv[])
{
    ofstream outFile("example.txt", ios::app);
    if (!outFile)
    {
        cerr << "Error opening file!" << endl;
        return 1;
    }
    outFile << "Appending line 1." << endl;
    outFile << "Appending line 2." << endl;
    outFile << "Appending line 3." << endl;
    outFile.close();
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-04.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ more example.txt
Hello, World!
This is a test file.
Appending line 1.
Appending line 2.
Appending line 3.
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

File I/O - 05

- Opening file in binary mode
- Writing bytes into a file.

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char *argv[])
{
    ofstream outFile("example.bin", ios::out | ios::binary);

    if (!outFile)
    {
        cerr << "Error opening file!" << endl;
        return 1;
    }

    char data[] = {0x01, 0x02, 0x03, 0x04};
    outFile.write(data, sizeof(data));

    char data1[] = {0x05, 0x06, 0x07, 0x08};
    outFile.write(data1, sizeof(data1));
    outFile.close();
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ more example.bin
```

File I/O - 06

- Opening file in NO TRUNCATE mode
- That is if a file exits, its contents will not be erased.

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char *argv[])
{
    fstream file("example.txt", ios::in | ios::out);

    if (!file)
    {
        cerr << "Error opening file!" << endl;
        return 1;
    }

    file.seekp(0, ios::end);
    file << "Adding line 1 without truncating." << endl;
    file << "Adding line 2 without truncating." << endl;
    file.close();

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ more example.txt
Hello, World!
This is a test file.
Appending line 1.
Appending line 2.
Appending line 3.
Adding line 1 without truncating.
Adding line 2 without truncating.
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$
```

File I/O - 07

- Detecting End of File

```
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char *argv[])
{
  ifstream in("data.txt");
  int roll_number = 0;
  char name[50], login[50];

  while( !in.eof() )
  {
    in >> roll_number >> name >> login;
    cout << "Roll Number: " << roll_number << endl;
    cout << "Name       : " << name << endl;
    cout << "Login      : " << login << endl;
  }
  in.close();
  return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ g++ t02-files-07.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/02$ ./a.out
Roll Number: 220101999
Name       : student-999
Login      : s999
Roll Number: 220101888
Name       : student-888
Login      : s888
Roll Number: 220101777
Name       : student-777
Login      : s777
Roll Number: 220101777
Name       : student-777
Login      : s777
```

Questions?

Thank You